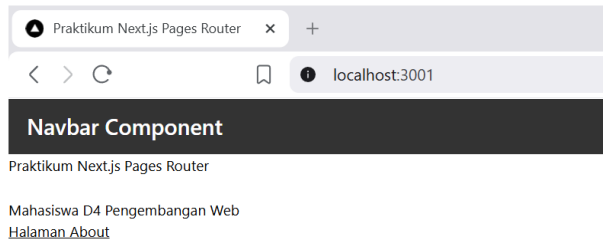


LAPORAN JOBSHEET 7: API ROUTES MATKUL PEMOGRAMAN BERBASIS NETWORK

AHMAD DZUL FADHLI HANNAN | TI3F | 2341720106

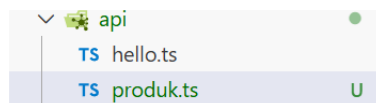
D. LANGKAH KERJA PRAKTIKUM

Langkah 1 – Menjalankan Project



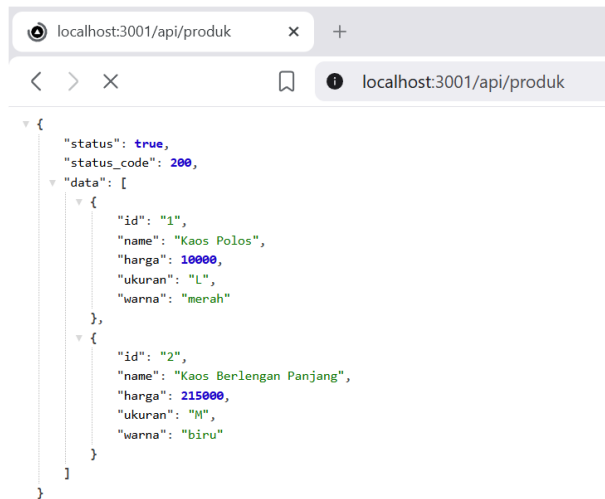
Langkah 2 – Membuat API Produk

Buat file pada pages/api/produk.js



```
pemograman_framework - produk.ts
1 // Next.js API route support: https://nextjs.org/docs/api-routes/introduction
2 import type { NextApiResponse, NextApiRequest } from "next";
3 type Data = {
4   status: boolean;
5   status_code: number;
6   data: {
7     id: string;
8     name: string;
9     harga: number;
10    ukuran: string;
11    warna: string;
12  };
13 };
14 export default function handler(
15   req: NextApiRequest,
16   res: NextApiResponse<Data>,
17 ) {
18   const data = [
19     {
20       id: "1",
21       name: "Kaos Polos",
22       harga: 10000,
23       ukuran: "L",
24       warna: "merah",
25     },
26     {
27       id: "2",
28       name: "Kaos Berlengan Panjang",
29       harga: 215000,
30       ukuran: "M",
31       warna: "biru",
32     },
33   ];
34   res.status(200).json({ status: true, status_code: 200, data: data });
35 }
```

Akses: <http://localhost:3000/api/produk>

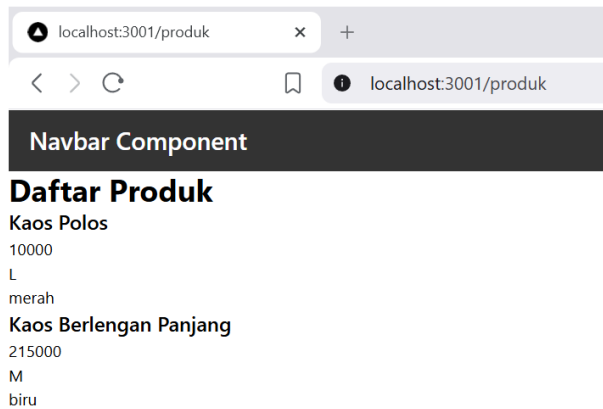


Langkah 3 – Fetch Data API di Frontend

Modifikasi kode pages/product/index.tsx



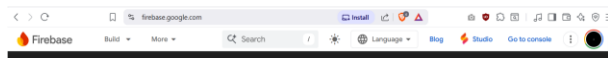
Jalankan browser <http://localhost:3000/produk>



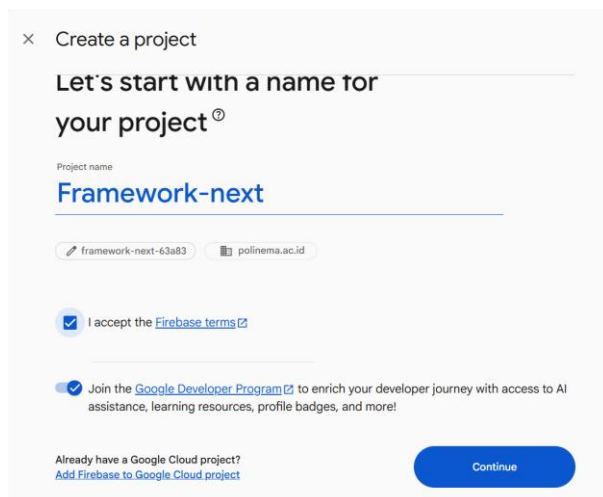
E. INTEGRASI FIREBASE

Langkah 5 – Setup Firebase

Buka Firebase Go To Console (login dengan login google)



Buat project baru



×

Create a project

Google Analytics

for your Firebase project

Google Analytics is a free and unlimited analytics solution that enables targeting, reporting, and more in Firebase Crashlytics, Cloud Messaging, In-App Messaging, Remote Config, A/B Testing, and Cloud Functions.

Google Analytics enables:

×

A/B testing ⓘ

×

Breadcrumb logs in Crashlytics ⓘ

×

User segmentation & targeting across Firebase products ⓘ

×

Event-based Cloud Functions triggers ⓘ

×

Free unlimited reporting ⓘ

☐

Enable Google Analytics for this project

Recommended

Previous

Create project



Framework-next

✓

Your Firebase project is ready

Continue

Try an agentic barista app powered by Firebase AI Logic!

Try Sample App

×

Framework-next

Spark plan

+ Add app

Try an agentic barista app powered by Firebase AI Logic!

Try Sample App

×

Framework-next

Spark plan

×













Select a platform

×

Add Firebase to your web app

1

Register app

App nickname

Framework-next2026

☐

Also set up **Firebase Hosting** for this app. [Learn more](#)

Hosting can also be set up later. There is no cost to get started anytime.

Register app

2

Add Firebase SDK

×

Add Firebase to your web app

✓

Register app

2

Add Firebase SDK

☒ Use npm
 ☐ Use a <script> tag

If you're already using [npm](#) and a module bundler such as [webpack](#) or [Rollup](#), you can run the following command to install the latest SDK ([Learn more](#)):

```
$ npm install firebase
```

Then, initialize Firebase and begin using the SDKs for the products you'd like to use.

```
// Import the functions you need from the SDKs you need
import { initializeApp } from "firebase/app";
// TODO: Add SDKs for Firebase products that you want to use
// https://firebase.google.com/docs/web/setup#available-libraries

// Your web app's Firebase configuration
const firebaseConfig = {
  apiKey: "AIzaSyB6C4kFR_vd4kLfJrxwNo88pD8ude9cHtw",
  authDomain: "framework-next-63a83.firebaseio.com",
  projectId: "framework-next-63a83",
  storageBucket: "framework-next-63a83.firebaseio.com",
  messagingSenderId: "235728255005",
  appId: "1:235728255005:web:907641aac129ed95a2aef4"
};

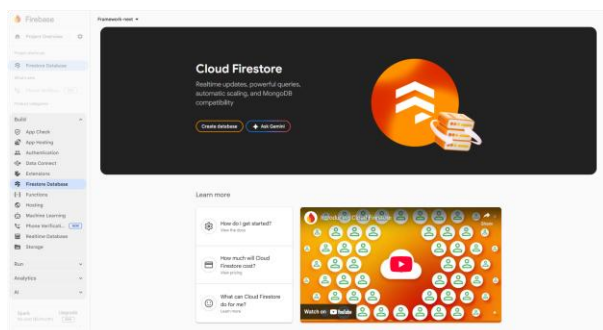
// Initialize Firebase
const app = initializeApp(firebaseConfig);
```

Note: This option uses the [modular JavaScript SDK](#), which provides reduced SDK size.

Learn more about Firebase for web: [Get Started](#), [Web SDK API Reference](#), [Samples](#)

Continue to console

Aktifkan Firestore Database



Create a database

- 1 Select edition

☒ **Standard edition**
 Simple query engine with automatic indexing
 Supports Core operations

☐ **Enterprise edition**
 Advanced query engine with self-managed indexing
 Supports Core, Pipelines, and MongoDB operations

Not sure which edition is right for you? [Compare editions](#)

Next
- 2 Database ID & location
- 3 Configure

- 1 Select edition
- 2 Database ID & location

Database ID

Location

Your location setting is where your Cloud Firestore data will be stored

After you set this location, you cannot change it later [Learn more](#)

Next
- 3 Configure

- 1 Select edition
- 2 Database ID & location
- 3 Configure

After you define your data structure, you will need to write rules to secure your data. [Learn more](#)

☒ **Start in production mode**
 Your data is private by default. Client read/write access will only be granted as specified by your security rules.

☐ **Start in test mode**
 Your data is open by default to enable quick setup. However, you must update your security rules within 30 days to enable long-term client read/write access.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

All third party reads and writes will be denied

Cancel **Create**

Firestore

Project Overview

Firestore Database

Rules
 Settings
 Database
 Security
 Indexes
 Collections

ID

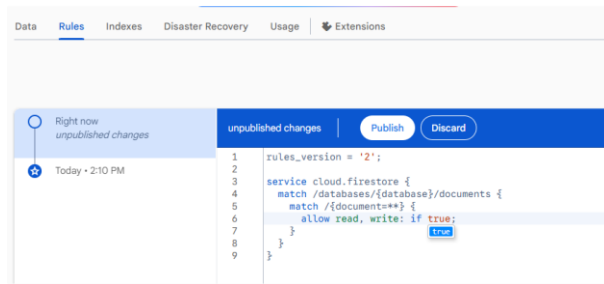
Index

Document

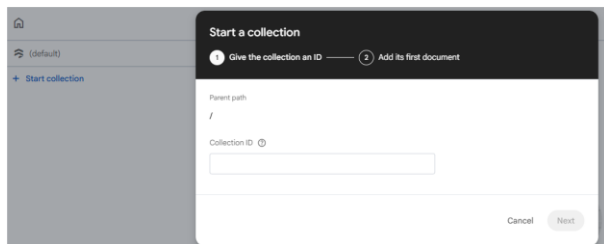
Your database is ready to go. Add and edit data.

Add new document

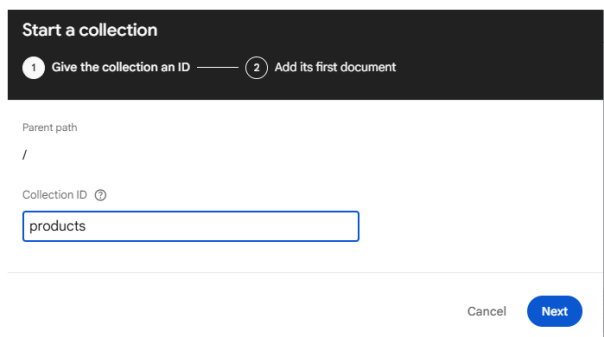
Rubah rulesnya jadi true dan klik publish



Buat collection:



Products



Gunakan auto-id

Start a collection

✓ Give the collection an ID

2 Add its first document

Document parent path

/products

Document ID ⓘ

Auto-ID

Required

Field

Type

string

⊖

String

⊕ Add field

Cancel

Save

Tambahkan field:

Start a collection

✓ Give the collection an ID

2 Add its first document

Document parent path

/products

Document ID ⓘ

ILjOOQRq5DrYrddI9yQL

Field

name

Type

string

⊖

String

baju koko

Field

price

Type

number

Value

100000

⊖

Field

size

Type

string

⊖

String

xl

⊕ Add field

Cancel

Save

Langkah 6 – Install Firebase

npm install firebase

```

PS D:\pemograman_framework\Code\praktikum\my-app> npm install firebase

added 79 packages, and audited 468 packages in 34s

154 packages are looking for funding
  run `npm fund` for details

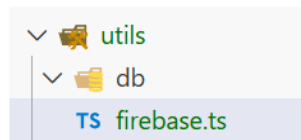
2 vulnerabilities (1 moderate, 1 high)

To address all issues, run:
  npm audit fix

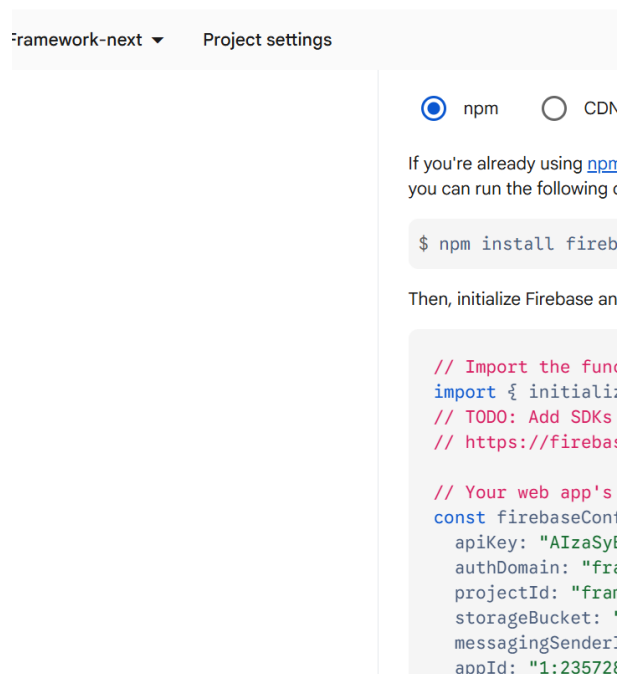
Run `npm audit` for details.
PS D:\pemograman_framework\Code\praktikum\my-app>

```

Buat folder dan file ts pada pages/utlis/db/firebase.ts

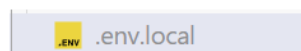


Copy paste yang ada pada kotak merah ke file firebase.ts



Langkah 7 – Konfigurasi Environment Variable agar credensial firebase tidak dapat dilihat saat dipush di repository

Buat file: .env.local



Modifikasi file env

Isikan sesuai dengan yang ada pada (tanpa koma dan petik dua)

Code > praktikum > my-app > .env.loc

```
1 FIREBASE_API_KEY=AIzaSyB6I
2 FIREBASE_AUTH_DOMAIN=frame
3 FIREBASE_PROJECT_ID=frame
4 FIREBASE_STORAGE_BUCKET=f
5 FIREBASE_MESSAGING_SENDE
6 FIREBASE_APP_ID=1:2357282
```

Langkah 8 – Konfigurasi Firebase

Modifikasi firebase.ts

```
pemograman_framework - firebase.ts
1 // Import the functions you need from the SDKs you need
2 import { initializeApp } from "firebase/app";
3 // TODO: Add SDKs for Firebase products that you want to use
4 // https://firebase.google.com/docs/web/setup#available-libraries
5
6 // Your web app's Firebase configuration
7 const firebaseConfig = {
8   apiKey: process.env.FIREBASE_API_KEY,
9   authDomain: process.env.FIREBASE_AUTH_DOMAIN,
10  projectId: process.env.FIREBASE_PROJECT_ID,
11  storageBucket: process.env.FIREBASE_STORAGE_BUCKET,
12  messagingSenderId: process.env.FIREBASE_MESSAGING_SENDE
13  appId: process.env.FIREBASE_APP_ID
14 };
15
16 // Initialize Firebase
17 const app = initializeApp(firebaseConfig);
18
19 export default app;
```

Langkah 9 – Ambil Data dari Firestore

Buat file ulits/db/servicefirebase.ts

modifikasi file servicefirebase.ts

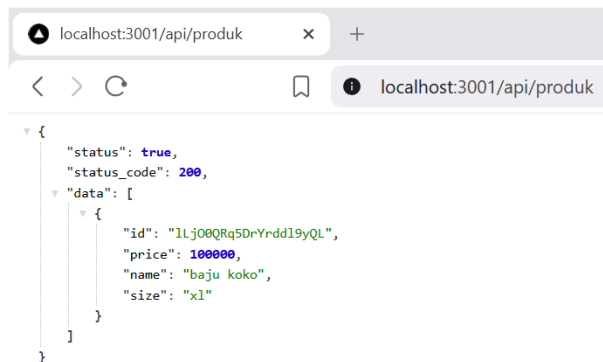
```
pemograman_framework - servicefirebase.ts
1 import {getFirestore, collection, getDocs} from 'firebase/firestore';
2 import app from './firebase';
3
4 const db = getFirestore(app);
5
6 export async function retrieveProducts(collectionName: string) {
7   const snapshot = await getDocs(collection(db, collectionName));
8   const data = snapshot.docs.map((doc)=>({
9     id: doc.id,
10     ...doc.data()
11   }));
12   return data;
13 }
14
```

Langkah 10 – API Mengambil Data Firebase

Edit pages/api/product.js:

```
pemograman_framework - produk.ts
1 // Next.js API route support: https://nextjs.org/docs/api-routes/introduction
2 import type { NextApiResponse, NextApiResponse } from "next";
3 import { retrieveProducts } from "../utils/serviceFirebase";
4 type Data = {
5   status: boolean;
6   status_code: number;
7   data: any;
8 };
9 export default async function handler(
10   req: NextApiResponse,
11   res: NextApiResponse<Data>,
12 ) {
13   const data = await retrieveProducts("products");
14   res.status(200).json({ status: true, status_code: 200, data: data });
15 }
```

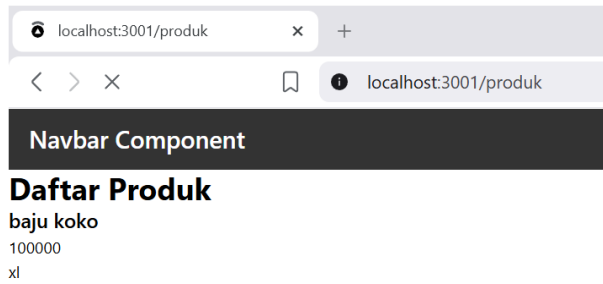
Jalankan browser <http://localhost:3000/api/produk>



Modifikasi index.ts pada produk sesuaikan nama typenya dan db nya

```
pemograman_framework - index.tsx
1 import { useRouter } from "next/router";
2 import { useEffect, useState } from "react";
3 // import TampilanProduk from "../views/produk";
4 type ProductType = {
5   id: string;
6   name: string;
7   price: number;
8   size: string;
9 };
10 const kategori = () => {
11   // const [isLoggedIn, setIsLoggedIn] = useState(false);
12   // const [push] = useRouter();
13   const [products, setProducts] = useState([]);
14   // useEffect(() => {
15   //   if(!isLoggedIn){
16   //     push("/auth/login"); //Terapkan redirect otomatis ke login jika user belum login.*/
17   //   }
18   // }, []);
19   useEffect(() => {
20     fetch("/api/produk")
21       .then((response) => response.json())
22       .then((responseData) => {
23         // console.log(responseData);
24         setProducts(responseData.data);
25       })
26       .catch((error) => {
27         console.error("Error fetching produk:", error);
28       })
29   }, []);
30   return (
31     <div>
32       <h1 className="text-3xl font-bold">Daftar Produk</h1>
33       <div>
34         {products.map((product: ProductType) => (
35           <div key={product.id}>
36             <h2 className="text-xl font-semibold">{product.name}</h2>
37             <p>{product.price}</p>
38             <p>{product.size}</p>
39           </div>
40         ))}
41       </div>
42     </div>
43   );
44 }
45 export default kategori;
```

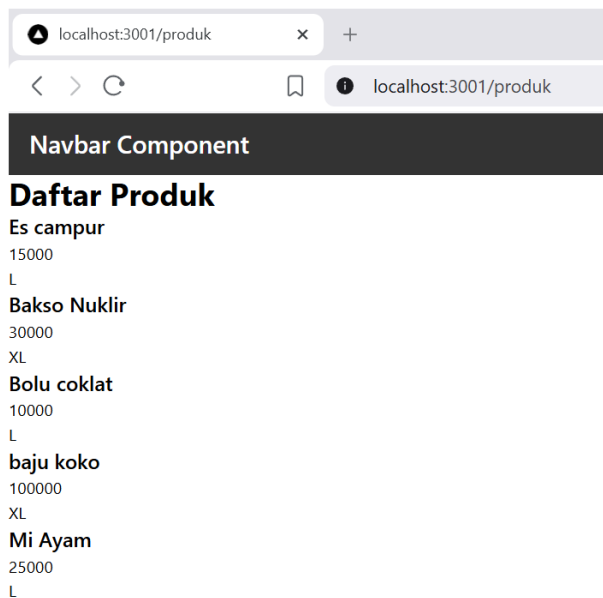
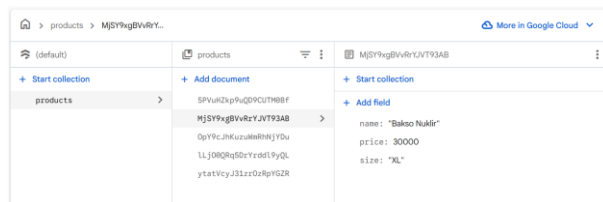
Jalankan browser



F. TUGAS PRAKTIKUM

Tugas 1 (Wajib)

- Tambahkan minimal 3 data produk di Firestore
- Pastikan data tampil di halaman produk



Tugas 2 (Wajib)

- Tambahkan field baru:
o category
- Tampilkan category di frontend

+ Add field

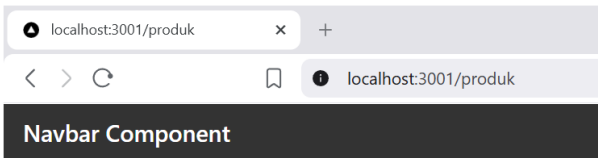
```
category: "makanan"

name: "Es campur"

price: 15000

size: "L"
```

```
pemograman_framework - index.tsx
1 import { useRouter } from "next/router";
2 import { useEffect, useState } from "react";
3 // import TampilanProduk from "../views/produk";
4 type ProductType = {
5   id: string;
6   name: string;
7   price: number;
8   size: string;
9   category: string;
10 }
11 const kategori = () => {
12   // const [isLoggedIn, setIsLoggedIn] = useState(false);
13   // const [push] = useRouter();
14   const [products, setProducts] = useState([]);
15   // useEffect(() => {
16   //   if(!isLoggedIn){
17   //     push("/auth/login");    /**Terapkan redirect otomatis ke login jika user belum login.**/
18   //   }
19   // }, []);
20   useEffect(() => {
21     fetch("/api/produk")
22       .then((response) => response.json())
23       .then((responseData) => {
24         // console.log(responseData);
25         setProducts(responseData.data);
26       })
27       .catch((error) => {
28         console.error("Error fetching produk:", error);
29       })
30   }, []);
31   return (
32     <div>
33       <h1 className="text-3xl font-bold">Daftar Produk</h1>
34       <products.map((product: ProductType) => {
35         <div key={product.id}>
36           <h2 className="text-xl font-semibold">Nama Produk: {product.name}</h2>
37           <p>Harga: Rp {product.price}</p>
38           <p>Ukuran: {product.size}</p>
39           <p>Kategori: {product.category}</p>
40         </div>
41       )}>
42     </div>
43   );
44 };
45 export default kategori;
```



Daftar Produk

Nama Produk: Es campur

Harga: Rp 15000

Ukuran: L

Kategori: minuman

Nama Produk: Bakso Nuklir

Harga: Rp 30000

Ukuran: XL

Kategori: makanan

Nama Produk: Bolu coklat

Harga: Rp 10000

Ukuran: L

Kategori: makanan

Nama Produk: baju koko

Harga: Rp 100000

Ukuran: XL

Kategori: pakaian

Nama Produk: Mi Ayam

Harga: Rp 25000

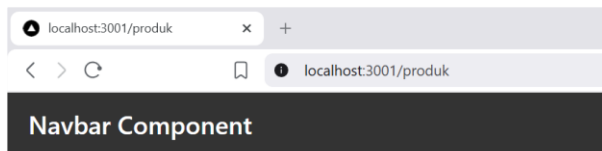
Ukuran: L

Kategori: makanan

Tugas 3 (Pengayaan)

- Tambahkan tombol Refresh Data
- Gunakan fetch ulang tanpa reload halaman

```
1 import { useRouter } from "next/router";
2 import { useEffect, useState } from "react";
3 // Import TampilanProduk from "../views/produk";
4 type ProductType = {
5   id: string;
6   name: string;
7   price: number;
8   size: string;
9   category: string;
10 }
11 const kategori = () => {
12   // const [isLoading, setIsLoading] = useState(false);
13   // const [push] = useRouter();
14   const [products, setProducts] = useState([]);
15   const [loading, setLoading] = useState(false);
16   const fetchProducts = async () => {
17     setLoading(true);
18     try {
19       const response = await fetch("/api/produk");
20       const responseData = await response.json();
21       setProducts(responseData.data);
22     } catch (error) {
23       console.error("Error fetching produk", error);
24     } finally {
25       setLoading(false);
26     }
27   };
28   useEffect(() => {
29     fetchProducts();
30   }, []);
31   // useEffect(() => {
32   //   if(!isLoading){
33   //     push("/auth/login"); //terapkan redirect otomatis ke login jika user belum login.*/
34   //   }
35   // }, []);
36   return (
37     <div className="container mx-auto p-4">
38       <h3 className="text-3xl font-bold">Daftar Produk</h3>
39       <button onClick={fetchProducts} className="mt-4 mb-4 py-2 bg-blue-500 text-white font-sans font-medium rounded hover:lg-blue-600">
40         {loading ? "Loading..." : "Refresh Produk"}
41       </button>
42       <div>
43         {products.map((product: ProductType) => {
44           <div key={product.id}>
45             <h4 className="text-xl font-sans">Nama Produk: {product.name}</h4>
46             <p>Harga: Rp {product.price}</p>
47             <p>Ukuran: {product.size}</p>
48             <p>Kategori: {product.category}</p>
49           </div>
50         })}
51       </div>
52     </div>
53   );
54 };
55 export default kategori;
```



Daftar Produk

Refresh Produk

Nama Produk: Es campur

Harga: Rp 15000

Ukuran: L

Kategori: minuman

Nama Produk: Bakso Nuklir

Harga: Rp 30000

Ukuran: XL

Kategori: makanan

G. PERTANYAAN EVALUASI

1. Apa fungsi API Routes pada Next.js?

Membuat endpoint backend langsung di dalam project Next.js tanpa server terpisah.

2. Mengapa .env.local tidak boleh di-push ke repository?

Karena biasanya berisi data rahasia seperti API key dan password.

3. Apa perbedaan data statis dan data dinamis?

Data yang tetap dan jarang berubah, sedangkan data dinamis adalah data yang dapat berubah sesuai waktu atau interaksi pengguna.

4. Mengapa Next.js disebut framework fullstack?

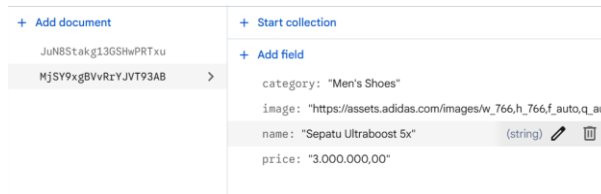
Karena bisa menangani frontend dan backend dalam satu project yang sama.

LAPORAN JOBSHEET 8: CLIENTSIDE RENDERING MATKUL PEMOGRAMAN BERBASIS NETWORK

AHMAD DZUL FADHLI HANNAN | TI3F | 2341720106

✂ C. LANGKAH PRAKTIKUM

Buka firebase dan rubah fieldnya serta tambahkan beberapa field seperti pada gambar dan untuk image bisa gunakan url pada image toko-toko sepatu (copy image address). Selain itu tambah juga documentnya sehingga ada 2 document



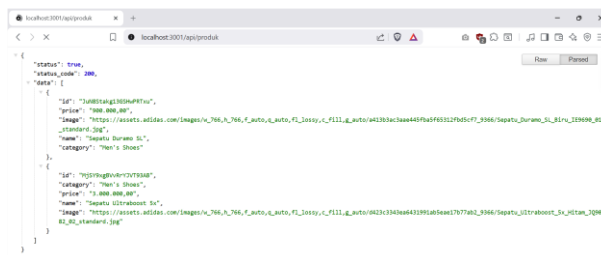
Bagian 1 – Setup Data Produk

Buat endpoint API /api/products.

Pastikan data memiliki:

- o id
- o name
- o category
- o price
- o image

jalankan browser <http://localhost:3000/api/produk>



Bagian 2 – Implementasi CSR dengan useEffect

Membuat file index.tsx pada folder views/products

Modifikasi index.tsx

```
pemograman_framework - index.tsx

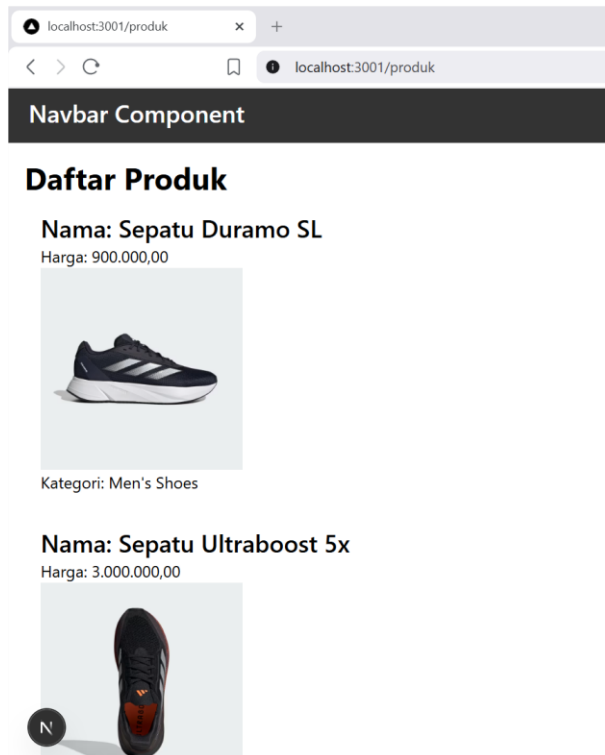
1  type ProductType = {
2    id: string;
3    name: string;
4    price: number;
5    image: string;
6    category: string;
7  };
8
9  const TampilanProduk = ({products}: {products: ProductType[]}) => {
10    return (
11      <div>
12        <h1 className="text-3xl font-bold">Daftar Produk</h1>
13        {products.map((product: ProductType) => (
14          <div className="p-4" key={product.id}>
15            <h2 className="font-semibold text-2xl">Nama: {product.name}</h2>
16            <p>Harga: {product.price}</p>
17            <img src={product.image} alt={product.name} width={200} />
18            <p>Kategori: {product.category}</p>
19          </div>
20        ))}
21      </div>
22    );
23  };
24  export default TampilanProduk;
```

Modifikasi index.tsx pada pages/produk/

```
pemograman_framework - index.tsx

1  import { useRouter } from "next/router";
2  import { useEffect, useState } from "react";
3  import TampilanProduk from "../views/product";
4  const kategori = () => {
5    // const [isLogin, setIsLogin] = useState(false);
6    // const {push} = useRouter();
7    const [products, setProducts] = useState([]);
8    useEffect(() => {
9      fetch("/api/produk")
10        .then((response) => response.json())
11        .then((responseData) => {
12          // setProducts(responseData.data);
13          console.log("Data Produk:", responseData.data);
14        })
15        .catch((error) => {
16          console.error("Error fetching produk:", error);
17        });
18    }, []);
19    return (
20      <div className="container mx-auto p-4">
21        <TampilanProduk products={products} />
22      </div>
23    );
24  };
25  export default kategori;
```

Jalankan browser <http://localhost:3000/produk>



Pada folder produk buat file produk.modules.scss

Modifikasi produk.modules.scss

pemograman_framework - produk.module.scss

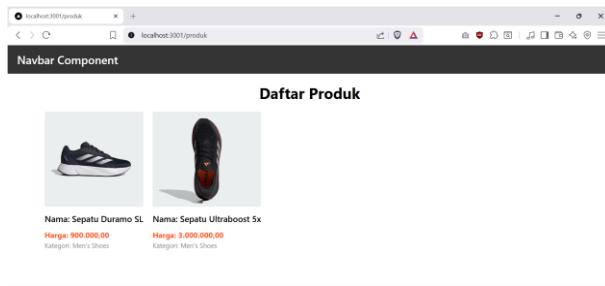
```
1 .produk {
2   width: 100%;
3   padding: 0 5%;
4   &__title {
5     text-align: center;
6     font-size: 32px;
7     font-weight: bold;
8     margin-bottom: 16px;
9   }
10  &__content {
11    display: flex;
12    flex-wrap: wrap;
13    gap: 20px;
14    &__item {
15      &__image {
16        width: 100%;
17        height: 200px;
18        object-fit: cover;
19        border-radius: 4px;
20        margin-bottom: 12px;
21      }
22      &__name {
23        font-size: 18px;
24        font-weight: 600;
25        margin-bottom: 8px;
26      }
27      &__price {
28        font-size: 16px;
29        color: #ff5722;
30        font-weight: bold;
31      }
32      &__category {
33        font-size: 14px;
34        color: #888;
35        margin-bottom: 8px;
36      }
37    }
38  }
39 }
40
```

Modifikasi Pada file index.tsx pada folder pages/views/product

pemograman_framework - index.tsx

```
1 import styles from "../../produk/produk.module.scss"
2 type ProductType = {
3   id: string;
4   name: string;
5   price: number;
6   image: string;
7   category: string;
8 };
9
10 const TampilanProduk = ({ products }: { products: ProductType[] }) => {
11   return (
12     <div className={styles.produk}>
13       <h1 className={styles.produk__title}>Daftar Produk</h1>
14       <div className={styles.produk__content}>
15         {products.map((product: ProductType) => (
16           <div className={styles.produk__content__item} key={product.id}>
17             <img src={product.image} alt={product.name} className={styles.produk__content__item__image} />
18             <h2 className={styles.produk__content__item__name}>Nama: {product.name}</h2>
19             <p className={styles.produk__content__item__price}>Harga: {product.price}</p>
20             <p className={styles.produk__content__item__category}>Kategori: {product.category}</p>
21           </div>
22         ))}
23       </div>
24     </div>
25   );
26 };
27
28 export default TampilanProduk;
```

Jalankan Browser



Bagian 3 – Implementasi Skeleton Loading

Modifikasi file index.tsx pada folder views/product/index.tsx

```

1 import styles from "../../produk/produk.module.scss"
2 type ProductType = {
3   id: string;
4   name: string;
5   price: number;
6   image: string;
7   category: string;
8 };
9
10 const TampilanProduk = ({ products }: { products: ProductType[] }) => {
11   return (
12     <div className={styles.produk}>
13       <h1 className={styles.produk__title}>Daftar Produk</h1>
14       <div className={styles.produk__content}>
15         {/* (products.map((product: ProductType) => {
16           <div className={styles.produk__content__item} key={product.id}>
17             <img src={product.image} alt={product.name} className={styles.produk__content__item__image} />
18             <h2 className={styles.produk__content__item__name}>Nama: {product.name}</h2>
19             <p className={styles.produk__content__item__price}>Harga: {product.price}</p>
20             <p className={styles.produk__content__item__category}>Kategori: {product.category}</p>
21           </div>
22         }))] */}
23         <div className={styles.produk__content__skeleton}>
24           <div className={styles.produk__content__skeleton__image}></div>
25           <div className={styles.produk__content__skeleton__name}></div>
26           <div className={styles.produk__content__skeleton__category}></div>
27           <div className={styles.produk__content__skeleton__price}></div>
28         </div>
29       </div>
30     </div>
31   );
32 };
33
34 export default TampilanProduk;

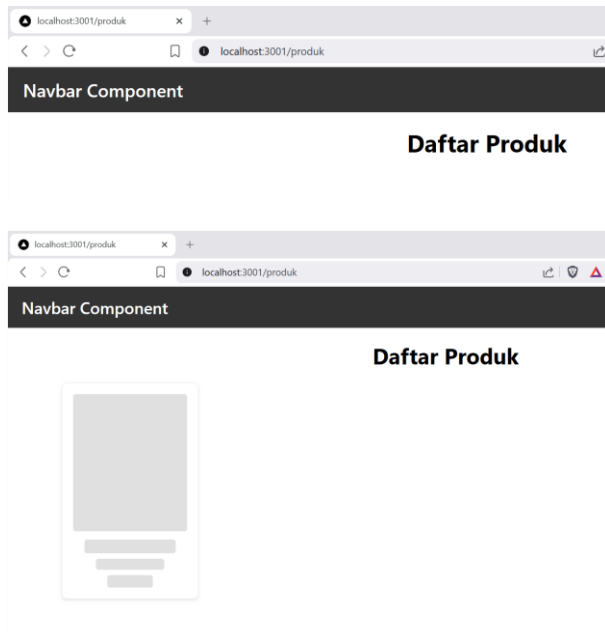
```

Modifikasi file product.module.scss

● ● ● pemograman_framework - produk.module.scss

```
1 .produk {
2   width: 100%;
3   padding: 0 5%;
4   &__title {
5     text-align: center;
6     font-size: 32px;
7     font-weight: bold;
8     margin-bottom: 16px;
9   }
10  &__content {
11    display: flex;
12    flex-wrap: wrap;
13    gap: 20px;
14    &__skeleton {
15      width: 200px;
16      padding: 16px;
17      border: 1px solid #eee;
18      border-radius: 8px;
19      box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
20      display: flex;
21      flex-direction: column;
22      align-items: center;
23      animation: identifier 1.5s infinite ease-in-out;
24      &__image {
25        width: 100%;
26        height: 200px;
27        background-color: #e0e0e0;
28        border-radius: 4px;
29        margin-bottom: 12px;
30      }
31      &__name {
32        width: 80%;
33        height: 20px;
34        background-color: #e0e0e0;
35        border-radius: 4px;
36        margin-bottom: 8px;
37      }
38      &__category {
39        width: 60%;
40        height: 16px;
41        background-color: #e0e0e0;
42        border-radius: 4px;
43        margin-bottom: 8px;
44      }
45      &__price {
46        width: 40%;
47        height: 18px;
48        background-color: #e0e0e0;
49        border-radius: 4px;
50      }
51    }
52    &__item {
53      &__image {
54        width: 100%;
55        height: 200px;
56        object-fit: cover;
57        border-radius: 4px;
58        margin-bottom: 12px;
59      }
60      &__name {
61        font-size: 18px;
62        font-weight: 600;
63        margin-bottom: 8px;
64      }
65      &__price {
66        font-size: 16px;
67        color: #ff5722;
68        font-weight: bold;
69      }
70      &__category {
71        font-size: 14px;
72        color: #888;
73        margin-bottom: 8px;
74      }
75    }
76  }
77 }
78
79 @keyframes identifier {
80   0% {
81     opacity: 1;
82   }
83   50% {
84     opacity: 0;
85   }
86   100% {
87     opacity: 1;
88   }
89 }
```

Jalankan browser maka akan muncul skeleton yang terdapat animasi berkedip

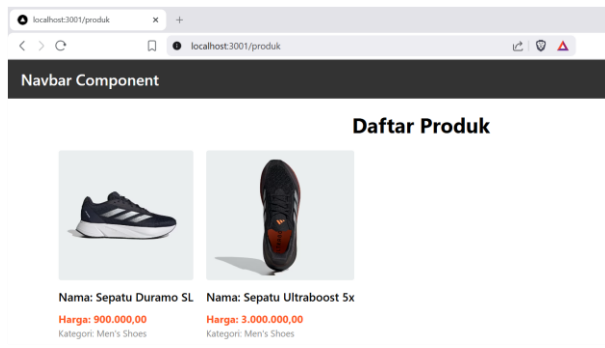


Modifikasi pada index.tsx pada folder views/product/index.tsx

```
perograman_framework - index.tsx
1 import styles from "../../produk/produk.module.scss"
2 type ProductType = {
3   id: string;
4   name: string;
5   price: number;
6   image: string;
7   category: string;
8 };
9
10 const TampilanProduk = ({ products }: { products: ProductType[] }) => {
11   return (
12     <div className={styles.produk}>
13       <h1 className={styles.produk_title}>Daftar Produk</h1>
14       <div className={styles.produk_content}>
15         {products.length > 0 ? (
16           <div>
17             {products.map((product: ProductType) => {
18               <div className={styles.produk_content_item} key={product.id}>
19                 <img src={product.image} alt={product.name} className={styles.produk_content_item_image} />
20                 <h2 className={styles.produk_content_item_name}>Nama: {product.name}</h2>
21                 <p className={styles.produk_content_item_price}>Harga: {product.price}</p>
22                 <p className={styles.produk_content_item_category}>Kategori: {product.category}</p>
23               </div>
24             )}
25           </div>
26         ) : (
27           <div className={styles.produk_content_skeleton}>
28             <div className={styles.produk_content_skeleton_image}></div>
29             <div className={styles.produk_content_skeleton_name}></div>
30             <div className={styles.produk_content_skeleton_category}></div>
31             <div className={styles.produk_content_skeleton_price}></div>
32           </div>
33         )}
34       </div>
35     </div>
36   );
37 }
38
39 export default TampilanProduk;
```

Jalankan browser

Jika dijalankan akan muncul skeletonnya terlebih dahulu setelah itu muncul gambar dan informasinya



Bagian 5 – Implementasi SWR

Install SWR

```
PS D:\pemograman_framework\Code\praktikum\my-app> npm install swr

added 3 packages, and audited 471 packages in 43s

154 packages are looking for funding
  run `npm fund` for details

2 vulnerabilities (1 moderate, 1 high)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
```

Buka dan modifikasi file index.tsx pada folder pages/product/

```
pemograman_framework - index.tsx

1 import { useRouter } from "next/router";
2 import { useEffect, useState } from "react";
3 import TampilanProduk from "../views/product";
4 import useSWR from "swr";
5 const fetcher = (url: string) => fetch(url).then((res) => res.json());
6 const kategori = () => {
7   // const [isLoggedIn, setIsLoggedIn] = useState(false);
8   // const {push} = useRouter();
9   const [products, setProducts] = useState([]);
10  const {data, error, isLoading} = useSWR("/api/produk", fetcher);
11
12  // cek apakah data, error, atau isLoading sudah benar
13  // console.log("Data:", data);
14  // console.log("Error:", error);
15  // console.log("Is Loading:", isLoading);
16
17  return (
18    <div className="container mx-auto p-4">
19      <TampilanProduk products={products} />
20    </div>
21  );
22 };
23 export default kategori;
```

Buat folder swr pada utils dan tambahkan file dengan nama fetcher.js

Modifikasi file fetcher.ts

```
pemograman_framework - fetcher.ts

1 const fetcher = (url: string) => fetch(url).then((res) => res.json());
2 export default fetcher;
```

Modifikasi file index.tsx pada folder pages/produk

```

pemograman_framework - index.tsx
1 import { useRouter } from "next/router";
2 import { useEffect, useState } from "react";
3 import TampilanProduk from "../views/product";
4 import useSWR from "swr";
5 import fetcher from "../utils/swr/fetcher";
6 // const fetcher = (url: string) => fetch(url).then((res) => res.json());
7 const kategori = () => {
8   // const [isLogin, setsLogin] = useState(false);
9   // const {push} = useRouter();
10  const [products, setProducts] = useState([]);
11  const {data, error, isLoading} = useSWR("/api/produk", fetcher);
12
13  // cek apakah data, error, atau isLoading sudah benar
14  // console.log("Data:", data);
15  // console.log("Error:", error);
16  // console.log("Is Loading:", isLoading);
17
18  return (
19    <div className="container mx-auto p-4">
20      <TampilanProduk products={isLoading ? [] : data.data} />
21    </div>
22  );
23 };
24 export default kategori;

```

Bandingkan:

- useEffect manual
- SWR

useEffect manual digunakan untuk mengambil data secara sederhana dan diatur sendiri, sedangkan SWR adalah cara yang lebih praktis karena otomatis menangani caching, loading, dan pembaruan data.

D. TUGAS PRAKTIKUM

Tugas Individu

1. Jelaskan perbedaan:

- o Client Side Rendering
- o Server Side Rendering
- o Static Site Generation

Client Side Rendering merender halaman di browser pengguna, **Server Side Rendering** merender halaman di server setiap ada request, sedangkan **Static Site Generation** menghasilkan halaman statis sekali saat build dan dikirim sebagai file siap pakai.

2. Buat halaman produk dengan:

- o Skeleton loading
- o Animasi

/views/product/index.tsx

```

pemograman_framework - index.tsx
1 import styles from "../../produk/produk.module.scss"
2 type ProductType = {
3   id: string;
4   name: string;
5   price: number;
6   image: string;
7   category: string;
8 };
9
10 const TampilanProduk = ({ products, isLoading }: { products: ProductType[], isLoading: boolean }) => {
11   return (
12     <div className={styles.produk}>
13       <h1 className={styles.produk_title}>Daftar Produk</h1>
14       <div className={styles.produk_content}>
15         {isLoading ? (
16           <>
17             {products.map((product: ProductType) => {
18               <div className={styles.produk_content_item} key={product.id}>
19                 <img src={product.image} alt={product.name} className={styles.produk_content_item_image} />
20                 <h2 className={styles.produk_content_item_name}>Nama: {product.name}</h2>
21                 <p className={styles.produk_content_item_price}>Harga: {product.price}</p>
22                 <p className={styles.produk_content_item_category}>Kategori: {product.category}</p>
23               </div>
24             ))}
25           </>
26         ) : (
27           <>
28             {[1, 2, 3].map((item) => {
29               <div className={styles.produk_content_skeleton}>
30                 <div className={styles.produk_content_skeleton_image}></div>
31                 <div className={styles.produk_content_skeleton_name}></div>
32                 <div className={styles.produk_content_skeleton_category}></div>
33                 <div className={styles.produk_content_skeleton_price}></div>
34               </div>
35             ))}
36           </>
37         )}
38       </div>
39     </div>
40   );
41 };
42
43 export default TampilanProduk;

```

/pages/produk/index.tsx

```

pemograman_framework - index.tsx
20 <TampilanProduk products={data?.data || []} isLoading={isLoading} />

```

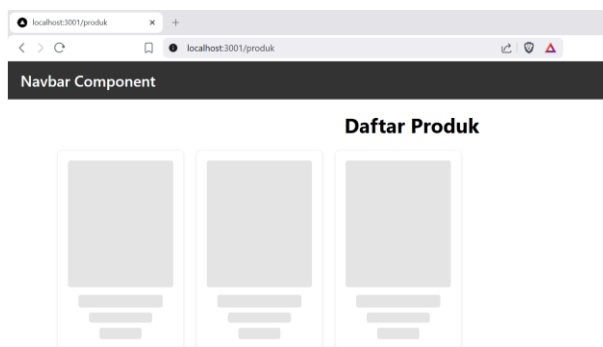
/pages/produk/index.tsx

```

pemograman_framework - produk.module.scss
83 50% {
84   opacity: 0.4;

```

Hasil



Menampilkan skeleton loading dengan animasi ease-in-out.

3. Refactor kode dari useEffect menjadi SWR.

/pages/produk.idex.tsx

```

1 import { useRouter } from "next/router";
2 import { useEffect, useState } from "react";
3 import TampilanProduk from "../views/product";
4 import useSWR from "swr";
5 import fetcher from "../utils/swr/fetcher";
6 // const fetcher = (url: string) => fetch(url).then((res) => res.json());
7 const kategori = () => {
8   // const [isLogin, setIsLogin] = useState(false);
9   // const {push} = useRouter();
10  const [products, setProducts] = useState([]);
11  // Menggunakan SWR
12  const {data, error, isLoading} = useSWR("/api/produk", fetcher);
13
14  // cek apakah data, error, atau isLoading sudah benar
15  // console.log("Data:", data);
16  // console.log("Error:", error);
17  // console.log("Is Loading:", isLoading);
18
19  return (
20    <div className="container mx-auto p-4">
21      <TampilanProduk products={data?.data || []} isLoading={isLoading} />
22    </div>
23  );
24 };
25 export default kategori;
```

LAPORAN JOBSHEET 9: SERVERSIDE RENDERING MATKUL PEMOGRAMAN BERBASIS NETWORK

AHMAD DZUL FADHLI HANNAN | TI3F | 2341720106

✂ C. LANGKAH PRAKTIKUM

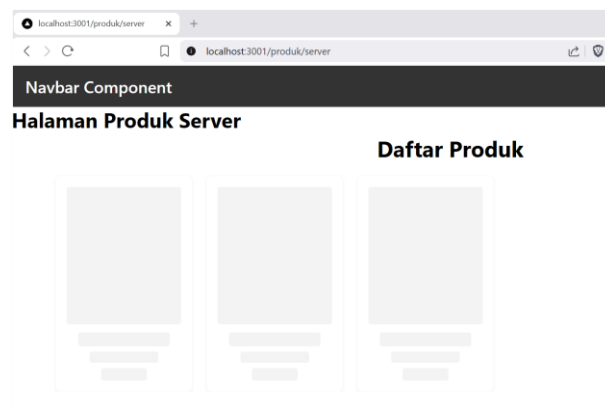
Bagian 1 – Setup Halaman SSR

Buat file baru pada pages/products/server.tsx

Modifikasi file server.tsx :

```
pemograman_framework - server.tsx
1 import TampilanProduk from "../views/product";
2 const halamanProdukServer = () => {
3   return (
4     <div>
5       <h1 className="font-bold text-3xl">Halaman Produk Server</h1>
6       <TampilanProduk products={[]} />
7     </div>
8   )
9 }
10 export default halamanProdukServer;
```

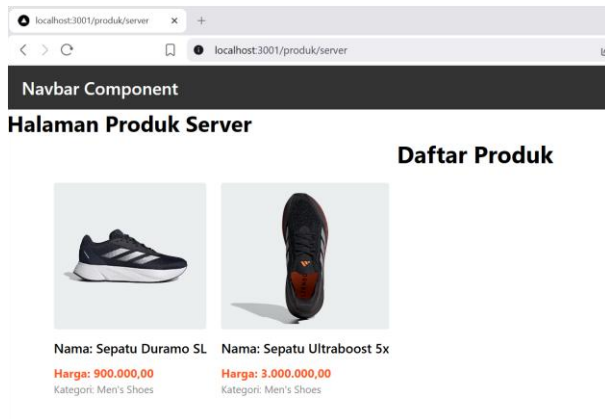
Jalankan browser : <http://localhost:3000/produk/server>



Bagian 2 – Implementasi getServerSideProps pada server.tsx

```
pemograman_framework - server.tsx
1 import TampilanProduk from "../views/product";
2
3 type ProductType = {
4   id: string;
5   name: string;
6   price: number;
7   image: string;
8   category: string;
9 };
10
11 const halamanProdukServer = (props: { products: ProductType[] }) => {
12   const { products } = props;
13   return (
14     <div>
15       <h1 className="font-bold text-3xl pl-4">Halaman Produk Server</h1>
16       <TampilanProduk products={products}/>
17     </div>
18   )
19 }
20 export default halamanProdukServer;
21
22 // Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil data produk dari API sebelum merender halaman.
23 export async function getServerSideProps() {
24   const res = await fetch("http://localhost:3001/api/produk");
25   const response = await res.json();
26   // console.log("Data produk yang diambil dari API", response);
27   return {
28     props: {
29       products: response.data,
30     },
31   }
32 }
33
```

Jalankan browser <http://localhost:3000/produk/server>



Bagian 3 – Refactor Type (produk type)

Buat folder types pada folder pages dan buat file Product.type.ts

Modifikasi Product.type.ts

```

1  export type ProductType = {
2    id: string;
3    name: string;
4    price: number;
5    image: string;
6    category: string;
7  }

```

Setelah membuat file Product.type.ts maka modifikasi pada file server.tsx menjadi

```

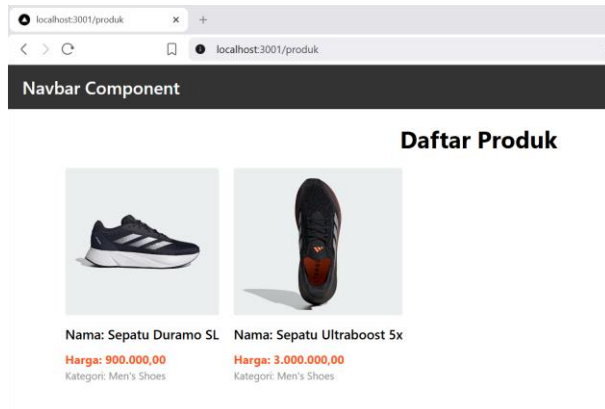
1  import TampilanProduk from "../views/product";
2  import { ProductType } from "../types/Product.type";
3
4  const halamanProdukServer = (props: { products: ProductType }) => {
5    const { products } = props;
6    return (
7      <div>
8        <h1 className="font-bold text-4xl">Halaman Produk Server</h1>
9        <TampilanProduk products={products}/>
10      </div>
11    );
12  }
13  export default halamanProdukServer;
14
15  // Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil data produk dari API sebelum merender halaman.
16  export async function getServerSideProps() {
17    const res = await fetch("http://localhost:3001/api/produk");
18    const response = await res.json();
19    // console.log("Data produk yang diambil dari API", response);
20    return {
21      props: {
22        products: response.data,
23      },
24    };
25  }
26

```

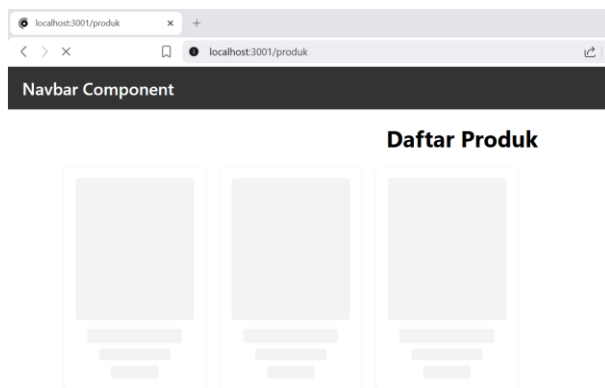
Bagian 4 – Uji Perbedaan SSR vs CSR

Uji 1 – Skeleton

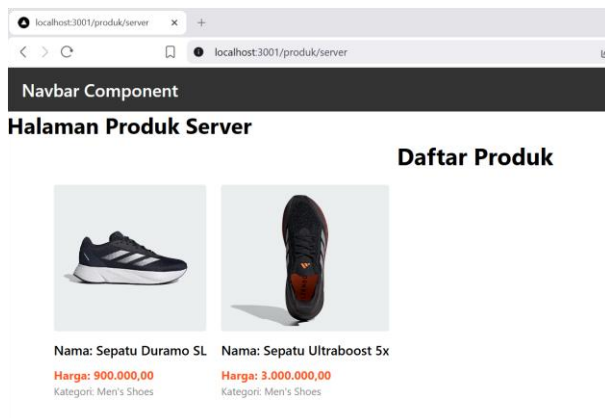
- Buka halaman CSR



- Refresh
- Skeleton muncul



- Buka halaman SSR



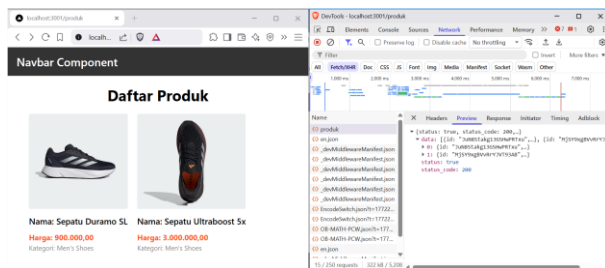
- Refresh
- Skeleton tidak muncul

Benar

Uji 2 – Network Tab

1. Buka DevTools → Network → XHR

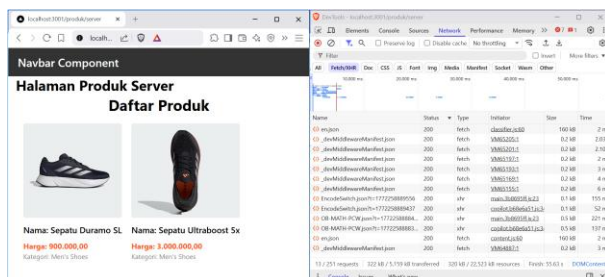
- 2 Refresh halaman CSR →
- Request API terlihat



Benar, terdapat data Request produk.

Refresh halaman SSR

→ Request API tidak terlihat



Benar, tidak ada Request produk.

Buat laporan analisis minimal 2 halaman

Hasil Analisis

Perbedaan antara **Client-Side Rendering (CSR)** dan **Server-Side Rendering (SSR)** terlihat dari bagaimana data produk ditampilkan di layar. Pada metode CSR yang diakses melalui URL /produk, browser berperan sebagai “pekerja utama” yang melakukan pengambilan data. Ketika halaman pertama kali dibuka, browser hanya menerima kerangka HTML kosong dari server. Setelah itu, kode JavaScript seperti useEffect atau SWR akan menjalankan perintah untuk mengambil data dari API Firebase. Karena proses pengambilan data ini membutuhkan

waktu, maka akan ditampilkan **Skeleton Loading** sebagai penanda bahwa aplikasi sedang bekerja di latar belakang.

Bisa dilihat lebih detail pada **Network Tab** di DevTools browser. Pada halaman CSR, muncul baris request baru bernama produk dengan tipe *fetch*. Hal ini menunjukkan bahwa ada aktivitas komunikasi data yang terjadi secara aktif di sisi klien (laptop/browser pengguna) setelah halaman berhasil dimuat. Keuntungannya adalah interaksi terasa lebih dinamis, namun kelemahannya adalah konten tidak langsung tersedia bagi mesin pencari (SEO) karena pada detik pertama perenderan, konten produk tersebut memang belum ada di dalam struktur HTML asli.

Sebaliknya, pada halaman SSR yang diakses melalui `/produk/server`, mekanismenya berubah total karena server Next.js mengambil alih seluruh tanggung jawab pengambilan data melalui fungsi `getServerSideProps`. Sebelum halaman dikirimkan ke browser pengguna, server sudah lebih dulu mengakses Firebase untuk menarik data produk, menyusunnya ke dalam komponen, dan menjadikannya menjadi dokumen HTML. Hasilnya, begitu halaman muncul di layar pengguna, data produk langsung tampil secara instan tanpa perlu melewati fase **Skeleton Loading** sama sekali.

Bisa dilihat lebih detail pada **Network Tab** browser. Tidak akan ada request `fetch` ke `/api/produk` karena proses tersebut sudah selesai dilakukan secara internal di "dalam" server sebelum HTML ditampilkan ke browser. Ini sangat menguntungkan untuk optimasi mesin pencari karena Google atau *crawler* lainnya dapat langsung membaca informasi produk (nama, harga, kategori) secara lengkap di dalam *source code* halaman tanpa harus menunggu eksekusi JavaScript terlebih dahulu.

Penggunaan library **SWR** pada sisi klien memberikan peningkatan signifikan dibandingkan penggunaan `useEffect` secara manual. SWR bekerja dengan prinsip *Stale-While-Revalidate*, di mana ia akan menampilkan data lama (cache) terlebih dahulu agar pengguna tidak merasa menunggu, sementara di latar belakang ia melakukan pengecekan ulang ke server apakah ada perubahan data terbaru. Kombinasi antara pemahaman kapan harus menggunakan SSR untuk kebutuhan SEO dan CSR/SWR untuk kebutuhan interaktivitas inilah yang menjadi kunci utama dalam membangun aplikasi web yang modern dan berperforma tinggi.

E. STUDI ANALISIS

Jawab pertanyaan berikut:

1. Mengapa SSR lebih baik untuk SEO?

Karena server mengirimkan dokumen HTML yang sudah berisi konten lengkap (seperti nama dan harga produk), sehingga mesin pencari dapat langsung mengindeks data tersebut tanpa harus menunggu eksekusi JavaScript.

2. Kapan sebaiknya menggunakan SSR?

Digunakan pada halaman publik yang memerlukan visibilitas tinggi di mesin pencari (SEO) dan konten yang datanya harus selalu segar atau sering berubah setiap kali diakses.

3. Apa kekurangan SSR dibanding CSR?

Adanya beban kerja yang lebih berat pada server karena harus merender ulang halaman di setiap request, serta latency yang lebih lama sebelum browser mulai menampilkan konten.

4. Mengapa skeleton tidak muncul pada SSR?

Karena seluruh proses pengambilan data sudah diselesaikan di sisi server sebelum halaman dikirim, sehingga saat browser menerima HTML, data sudah langsung ada untuk ditampilkan.